

o o o o

# Calitate și Testare Software

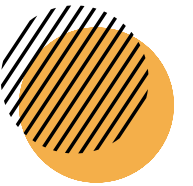
C10 – Strategy, Observer, Command

o o o o





# DP Comportamentale



Sunt șabloane de proiectare care descriu cum comunică obiectele între ele și cum este distribuită responsabilitatea pentru comportamente.

- Strategy (\*\*)
- Observer (\*\*)
- Chain of Responsibility (\*)
- Command (\*\*)
- Memento (\*\*\*)
- State (\*\*\*)
- Template (\*\*\*)



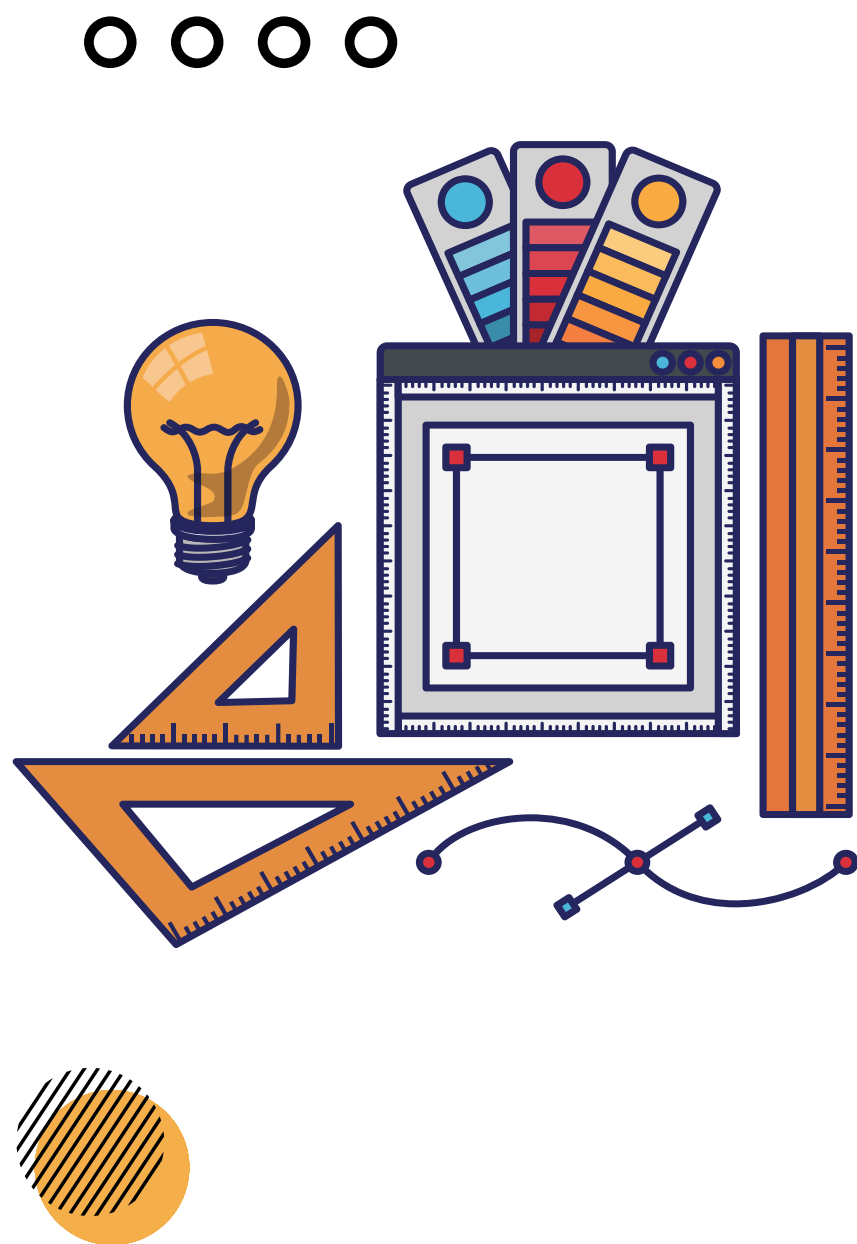


# Strategy



- Este folosit atunci când avem mai mulți algoritmi pentru rezolvarea unei probleme, iar alegerea implementării se realizează la run-time.
- Fiecare comportament este dat de o clasă.
- Definește strategia adoptată la run-time.





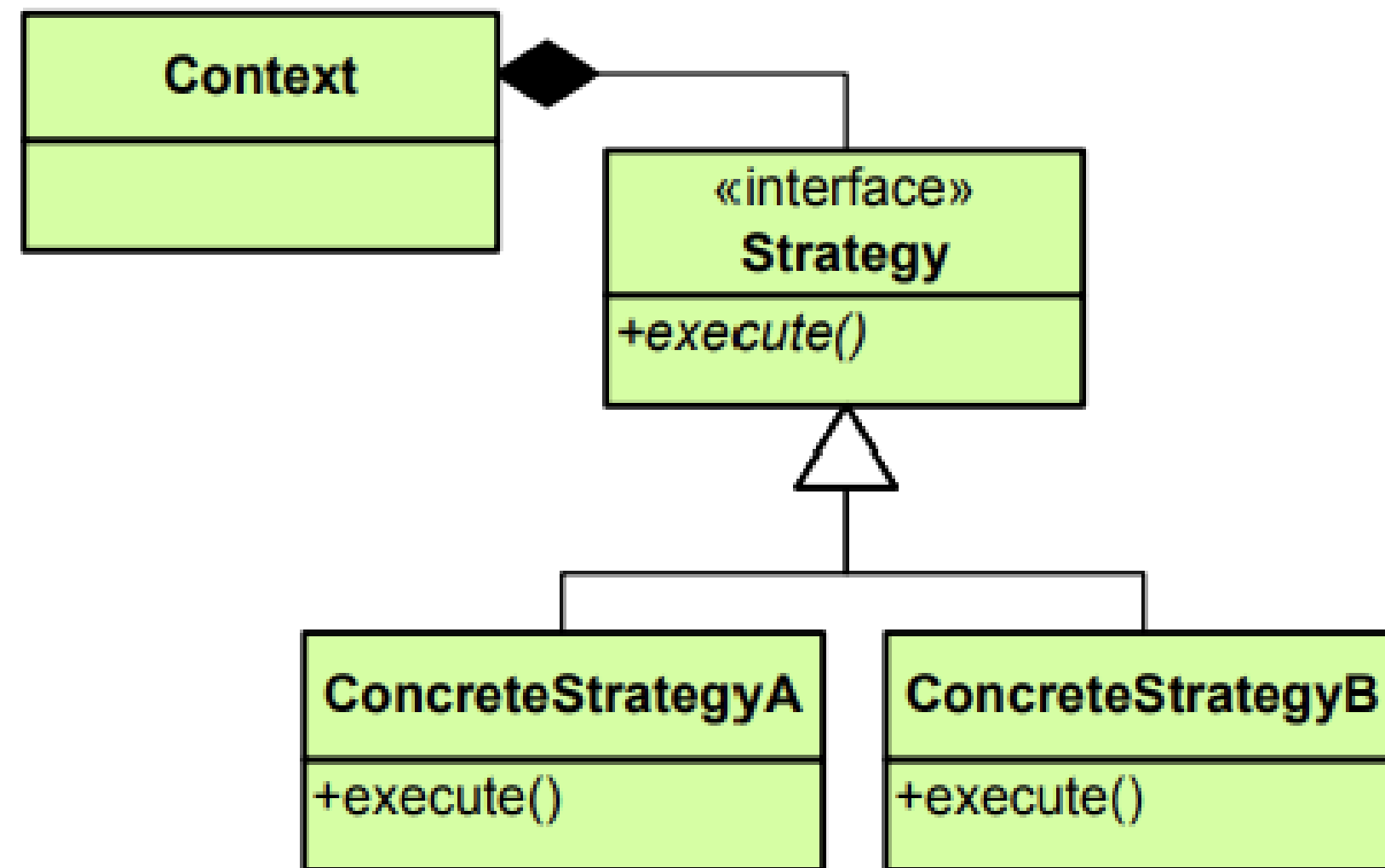
## Strategy – cerință

În cadrul restaurantului, pentru fiecare produs disponibil în cadrul meniului există informații cu referire la numărul de calorii, carbohidrați și alți macronutrienți. Clientul are posibilitatea să ceară ospătarului să îi aleagă un meniu ținând cont de regimul pe care acesta îl urmează: minimizare de calorii sau minimizare de carbohidrați.

Să se implementeze modulul ce oferă posibilitatea clientului de a-și alege tehnica de selecție a unui produs din lista de produse disponibile în meniu.



# Strategy – Diagrama



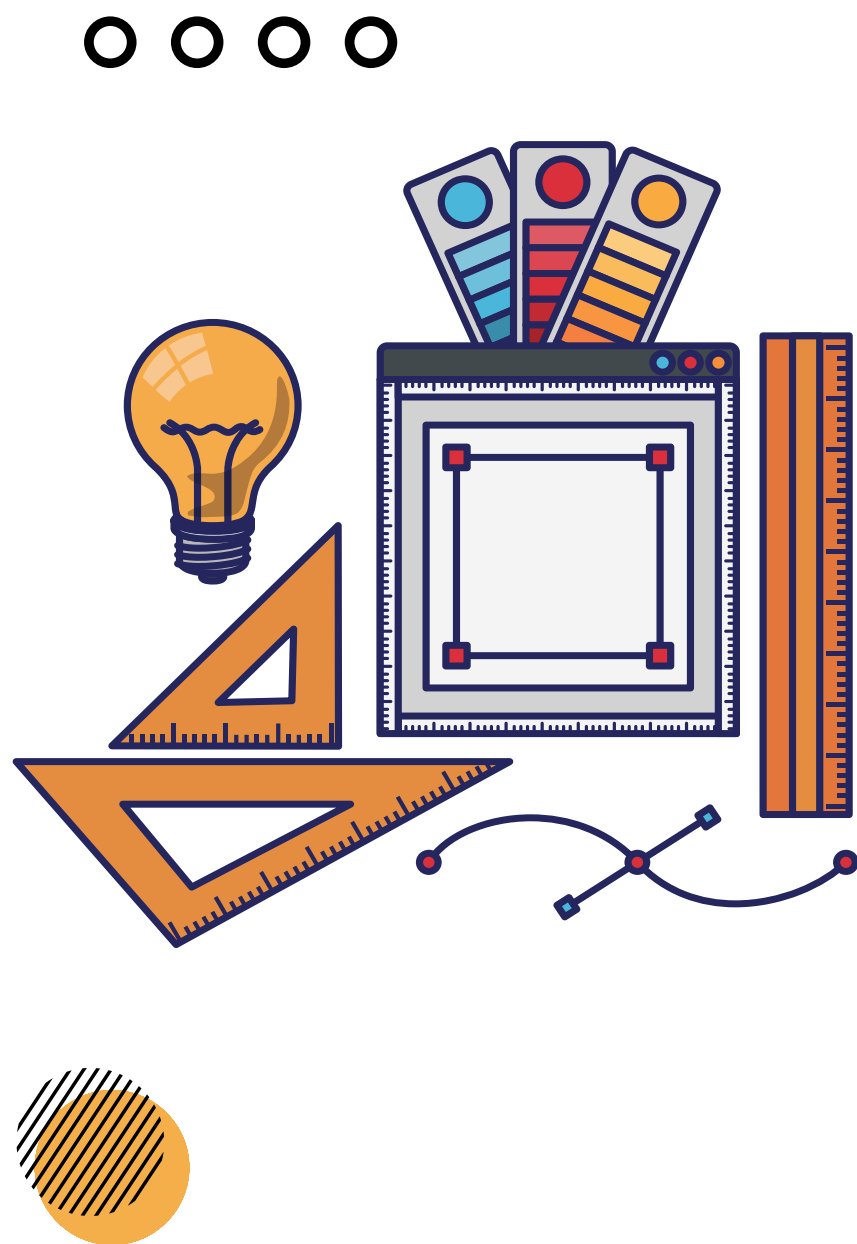


# Observer



- Observer definește o relație de “1 : n”, în care un subiect notifică mai mulți observari.
- Acest design pattern este folosit atunci când anumite elemente (obiecte) trebuie să fie anunțate de schimbarea stării altor obiecte.

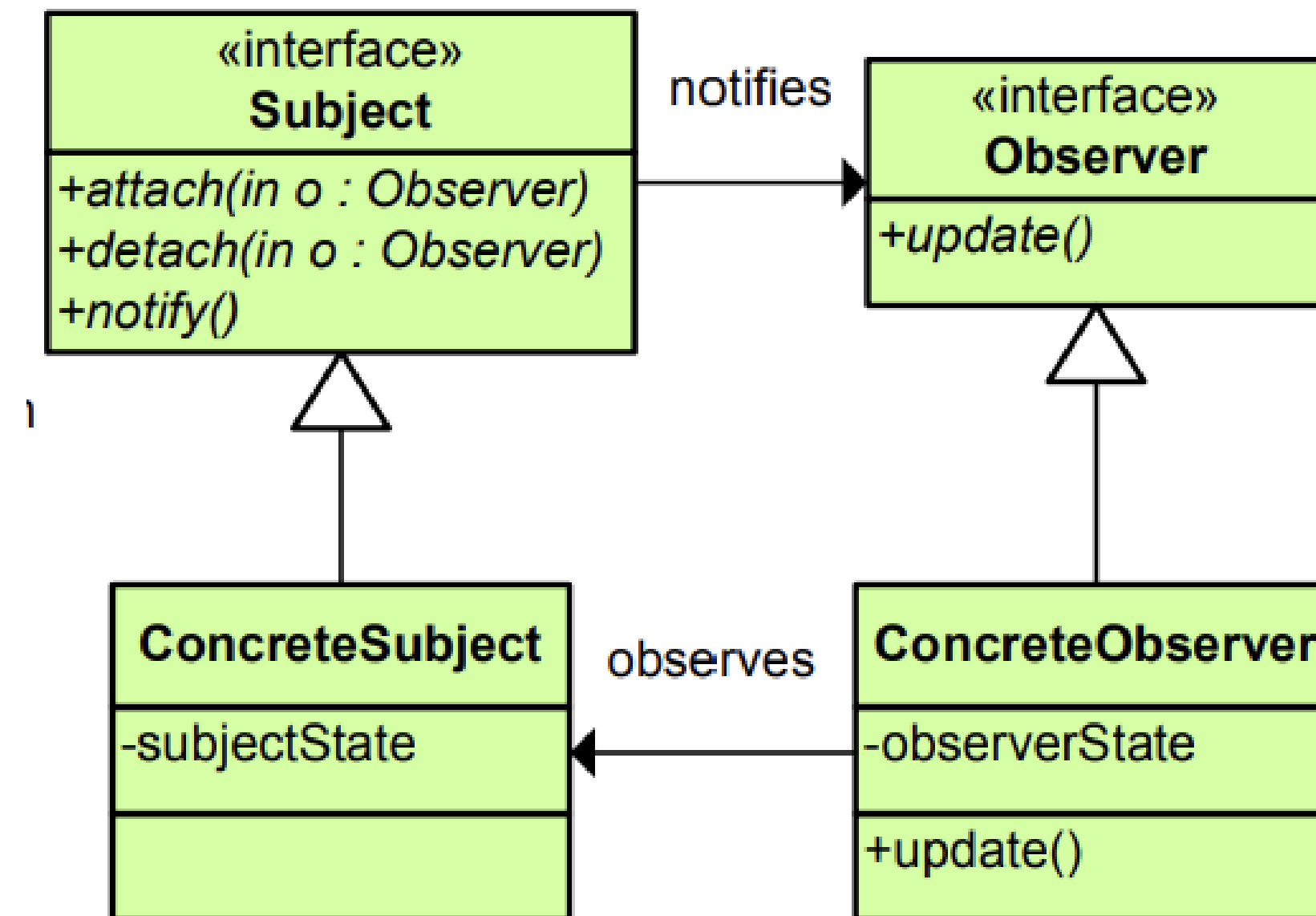




## Observer – cerință

Restaurantul dorește să anunțe clienții fideli ori de câte ori apar noi oferte. Astfel, se dorește implementarea unui modul care, atunci când se realizează o ofertă de preț sau se introduce un nou meniu, să se trimită notificări tuturor clienților abonați la notificările restaurantului.

# Observer – Diagrama



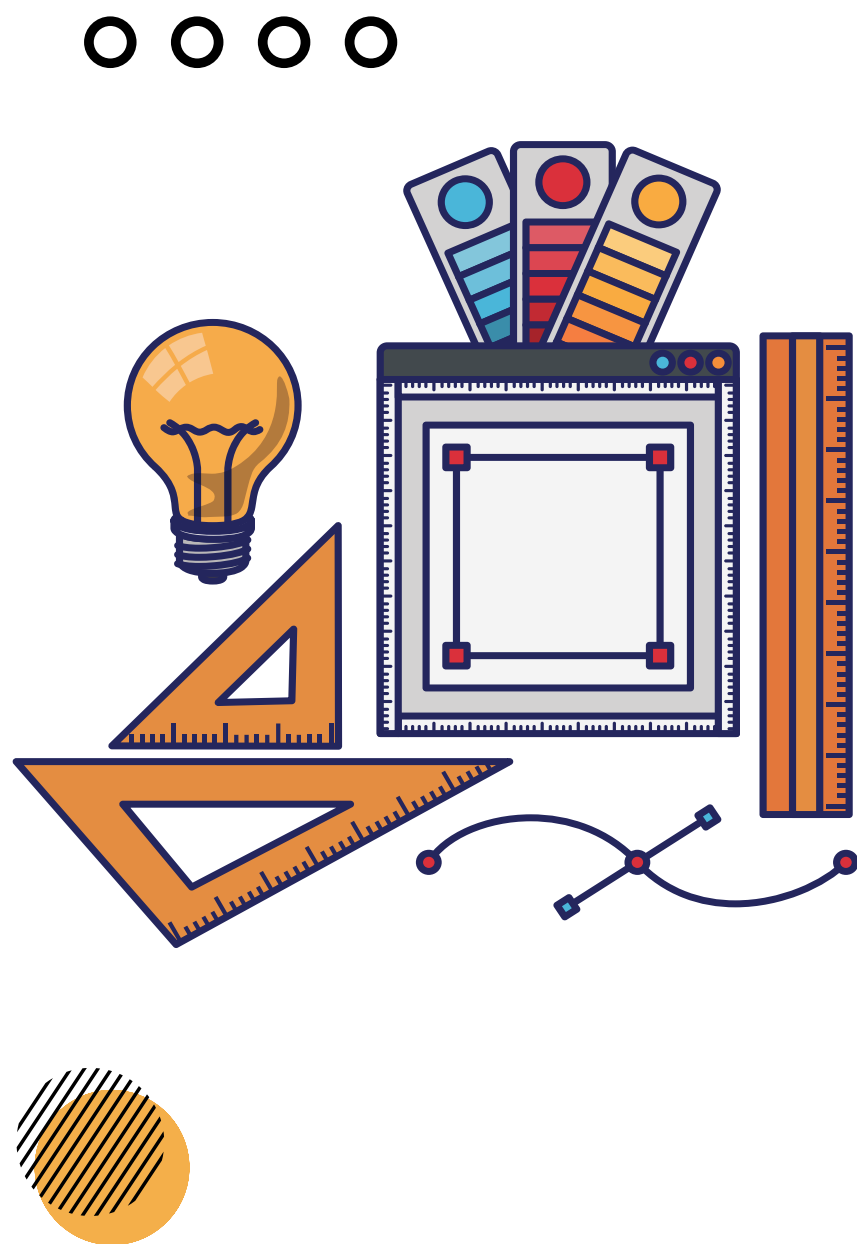


# Command



- Este folosit pentru implementarea **loose coupling**.
- In acest mod, se ascunde aplicarea de comenzi, fără a se ştie concret ce presupune acea comandă.
- Astfel clientul este decuplat de cel ce execută acţiunea.





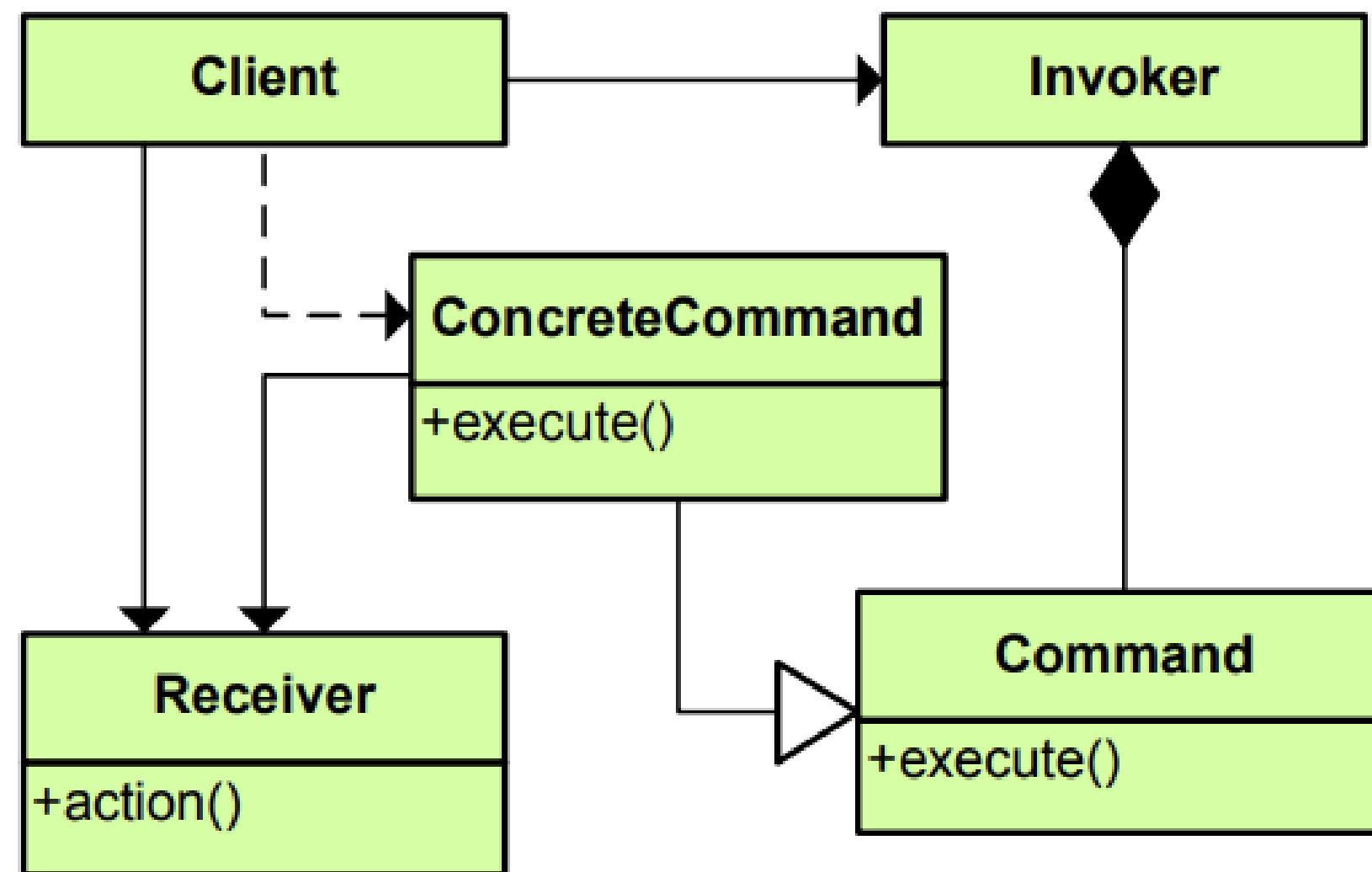
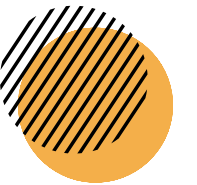
## Command – c e r i n ț ă

In cadrul restaurantului, ospatarul este responsabil sa preia comenzile de la clientii care sunt la masa. Dupa ce preia toate comenzile, le trimite la bucatarie, la bucatari, pentru a le putea prepara. Un bucatar poate prepara pizza sau paste. Comenzile astfel sunt de doua tipuri: comenzi pizza sau comenzi paste.

Sa se implementeze modulul ce gestioneaza preluarea comenzilor de catre ospatar si plasarea lor catre bucatari.

# Command – Diagrama

o o o o



o o o o



o o o o

# Next time



- Memento
- State
- Template



o o o o